

Serialization Format Taxonomy

- Pure serialization formats only
 - Includes text-based, binary JSON, schema-based binary, zero-copy
 - XML included in text-based serialization
-

1. Text-Based Serialization Formats (JSON, YAML, XML)

Description

- Human-readable structured text formats used for general-purpose data serialization.

Advantages

- Easy to inspect and debug
- Universal language support
- Flexible and expressive
- XML supports strong schema (XSD) and namespaces

Limitations

- Slow parsing
- Large payload size
- No strict schema by default (JSON/YAML)
- YAML whitespace-sensitive; XML verbose

Best For

- Web APIs (JSON)
- Human-readable structured data
- Systems requiring schema validation (XML)

Not For

- High-performance RPC
 - Low-bandwidth environments
 - Real-time systems
-

2. Binary JSON Serialization Formats (MessagePack, CBOR)

Description

- Compact binary encodings of JSON-like structures with flexible schemas.

Advantages

- Faster than JSON
- Smaller payloads
- Supports dynamic structures
- Good cross-language support

Limitations

- Not human-readable
- Tooling weaker than JSON/XML
- Requires libraries for browser use

Best For

- Microservices internal communication
- WebSocket messaging
- IoT and mobile devices

Not For

- Public APIs requiring readability
 - Schema-heavy systems
 - Analytical workloads
-

3. Schema-Based Binary Serialization Formats (Protobuf, Avro, Thrift)

Description

- Binary formats requiring predefined schemas (IDL), optimized for compactness and cross-language RPC.

Advantages

- Very compact binary representation
- High performance
- Strong schema evolution (Avro)
- Excellent for RPC

Limitations

- Not human-readable
- Requires schema management
- Debugging more difficult

Best For

- gRPC
- Microservices
- Cross-language communication
- Event streaming (Avro)

Not For

- Human-edited data
 - Zero-allocation real-time systems
 - Large analytical datasets
-

4. Zero-Copy Serialization Formats (FlatBuffers, Cap'n Proto)

Description

- Formats designed for extremely fast access without deserialization; data is directly readable in-place.

Advantages

- Zero-copy reading
- Extremely fast
- Very low memory overhead
- Random access to fields

Limitations

- More complex APIs
- Harder to debug
- Larger than Protobuf in some cases
- Weaker schema evolution

Best For

- Game engines
- Mobile apps
- Embedded systems
- Real-time rendering

Not For

- Traditional RPC
- Human-readable data
- Schema-evolving systems

Summary Table

Category	Description	Formats	Best For	Not For
Text-Based	Human-readable structured text	JSON, YAML, XML	Web, readable data	High-performance RPC
Binary JSON	Compact binary JSON	MessagePack, CBOR	IoT, microservices	Public APIs
Schema-Based Binary	IDL-based binary formats	Protobuf, Avro, Thrift	RPC, pipelines	Human editing
Zero-Copy	Direct-access binary layouts	FlatBuffers, Cap'n Proto	Games, embedded	Schema evolution